Edge Intelligence

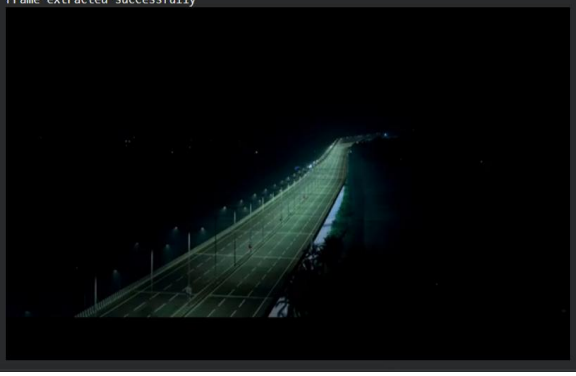
Video Proccesing

Name : Soni Samarth
Roll No : 25MML0004

File Link : https://drive.google.com/file/d/1RiiDiyNSvGXh5cwe6Cy0EciLU-in4gKL/view?usp=drive_link

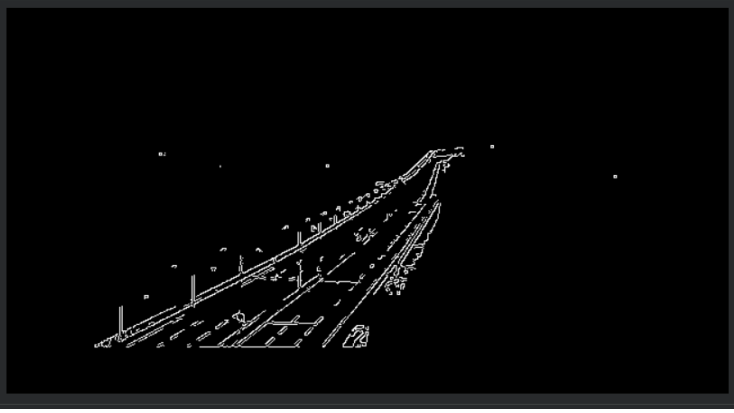
```
[1]
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import drive

[2]
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

[3]
video_path = "/content/drive/MyDrive/Edge_intelligence/videoplayback.mp4"
```

```
cap = cv2.VideoCapture(video_path)

ret, frame = cap.read()

if ret:
    print("Frame extracted successfully")
    cv2_imshow(frame)

cap.release()

Frame extracted successfully
```



```
[5]
0s
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

cv2_imshow(gray)
```

```
edges = cv2.Canny(gray, 100, 200)

cv2_imshow(edges)
```

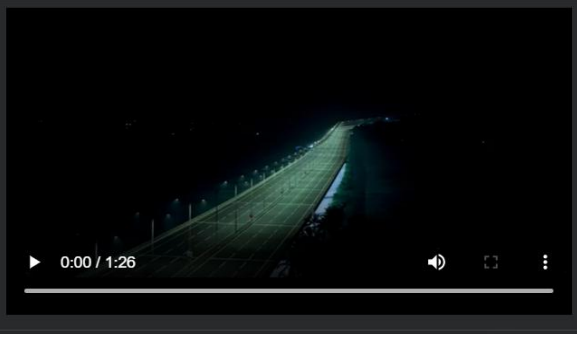


```python
def detect_video_edges(input_path, output_path):

    cap = cv2.VideoCapture(input_path)

    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    fps = cap.get(cv2.CAP_PROP_FPS)

    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    out = cv2.VideoWriter(output_path, fourcc, fps, (width, height), False)

    while True:
        ret, frame = cap.read()

        if not ret:
            break

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        edges = cv2.Canny(gray, 100, 200)

        out.write(edges)

    cap.release()
    out.release()

    print("Video processing completed!")
```

```python
input_video = "/content/drive/MyDrive/Edge_intelligence/videoplayback.mp4"

output_video = "/content/drive/MyDrive/Edge_intelligence/output.mp4"

detect_video_edges(input_video, output_video)
```

```
Video processing completed!
```

```python
from IPython.display import HTML
from base64 import b64encode

# Correcting the path for output_video, assuming the directory is 'Edge_intelligence' (lowercase 'i')
output_video = "/content/drive/MyDrive/Edge_intelligence/videoplayback.mp4"

mp4 = open(output_video,'rb').read()
data_url = "data:video/mp4; base64," + b64encode(mp4).decode()

HTML("""
<video width=500 controls>
      <source src="%s" type="video/mp4">
</video>
""" % data_url)
```



2) capturing the photo from camera

Capturing the photo

```python
import cv2
import numpy as np
from google.colab import drive
from google.colab.patches import cv2_imshow
from google.colab.output import eval_js
from IPython.display import display, Javascript, HTML
from base64 import b64decode, b64encode
```

```python
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```python
def take_photo(filename='photo.jpg', quality=0.8):

    js = Javascript('''
    async function takePhoto(quality) {
      const div = document.createElement('div');
      const capture = document.createElement('button');
      capture.textContent = 'Capture';
      div.appendChild(capture);

      const video = document.createElement('video');
      video.style.display = 'block';
      const stream = await navigator.mediaDevices.getUserMedia({video: true});

      document.body.appendChild(div);
      div.appendChild(video);
      video.srcObject = stream;
      await video.play();

      google.colab.output.setIframeHeight(document.documentElement.scrollHeight, true);

      await new Promise((resolve) => capture.onclick = resolve);

      const canvas = document.createElement('canvas');
      canvas.width = video.videoWidth;
      canvas.height = video.videoHeight;
      canvas.getContext('2d').drawImage(video, 0, 0);

      stream.getVideoTracks()[0].stop();
      div.remove();
```

```
        return canvas.toDataURL('image/jpeg', quality);
      }
      ''')

      display(js)

      data = eval_js('takePhoto({})'.format(quality))
      binary = b64decode(data.split(',')[1])

      with open(filename, 'wb') as f:
          f.write(binary)

      return filename
```

```
photo = take_photo()
```

```
img = cv2.imread(photo)
cv2_imshow(img)
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

cv2_imshow(gray)
```

```
print("Pixel value at (100,100): ", gray[100,100])
```

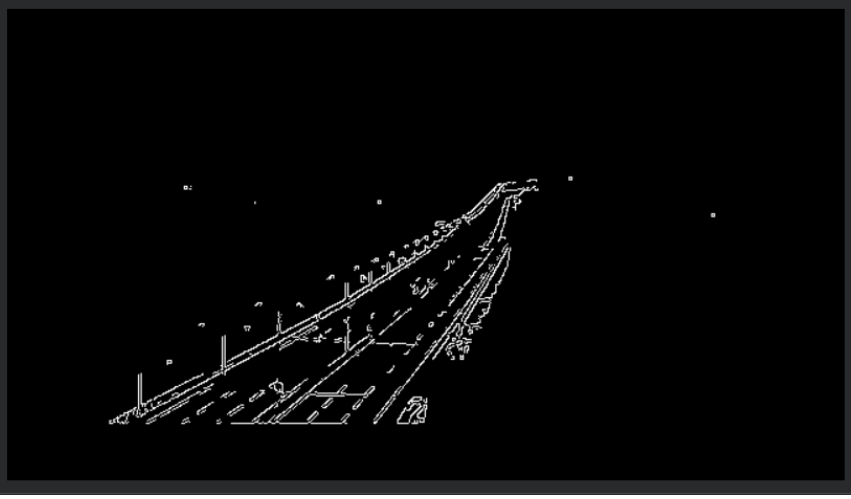
```
Pixel value at (100,100):  235
```

```
edges = cv2.Canny(gray, 100, 200)

cv2_imshow(edges)
```

3) frame extraction from video

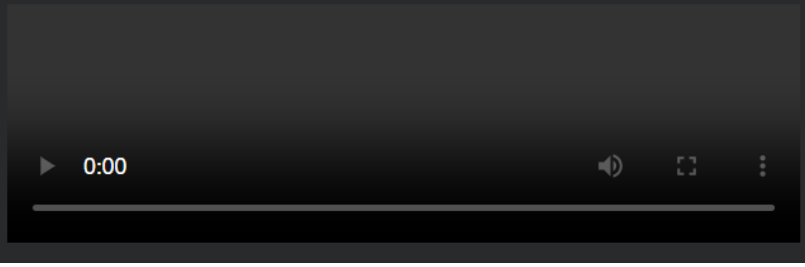
```
video_path = "/content/drive/MyDrive/Edge_intelligence/videoplayback.mp4"

cap = cv2.VideoCapture(video_path)

ret, frame = cap.read()

cv2_imshow(frame)

cap.release()
```

```python
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

edges = cv2.Canny(gray, 100, 200)

cv2_imshow(edges)
```



```python
def detect_video_edges(input_video, output_video):

    cap = cv2.VideoCapture(input_video)

    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    fps = cap.get(cv2.CAP_PROP_FPS)

    fourcc = cv2.VideoWriter_fourcc(*'mp4v')

    out = cv2.VideoWriter(output_video, fourcc, fps, (width, height), False)

    while True:

        ret, frame = cap.read()

        if not ret:
            break

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        edges = cv2.Canny(gray,100,200)

        out.write(edges)

    cap.release()
    out.release()

    print("Video processing completed")
```

```python
input_video = "/content/drive/MyDrive/Edge_intelligence/videoplayback.mp4"

output_video = "/content/drive/MyDrive/Edge_intelligence/edge_output.mp4"

detect_video_edges(input_video, output_video)
```

```
Video processing completed
```

```python
mp4 = open(output_video,'rb').read()

data_url = "data:video/mp4; base64," + b64encode(mp4).decode()

HTML("""
<video width=500 controls>
      <source src="%s" type="video/mp4">
</video>
""" % data_url)
```